

mlproject1.py

```
1 import pickle
2 import numpy as np
3 from keras.preprocessing.sequence import pad_sequences
4 from keras.preprocessing.text import Tokenizer
5 from keras.models import Sequential, Model
6 from keras.layers import Embedding, Input, Activation, Dense, Permute, Dropout,
concatenate, add, dot, LSTM
7 import matplotlib.pyplot as plt
8 with open("train_qa.txt", "rb") as fp:
9     train_data = pickle.load(fp)
10 with open("test_qa.txt", "rb") as fp:
11     test_data = pickle.load(fp)
12 vocab = set()
13 all_data = train_data + test_data
14 for story, ques, ans in all_data:
15     vocab = vocab.union(set(story))
16     vocab = vocab.union(set(ques))
17 vocab.add('yes')
18 vocab.add('no')
19 vocab_len = len(vocab) + 1
20 max_story_len = max([len(data[0]) for data in all_data])
21 max_ques_len = max([len(data[1]) for data in all_data])
22 tokenizer = Tokenizer(filters=[])
23 tokenizer.fit_on_texts(vocab)
24 def vectorize_data(data, word_index=tokenizer.word_index, max_story_len=max_story_len,
max_ques_len=max_ques_len):
25     X = []
26     Xq = []
27     A = []
28     for story, ques, ans in data:
29         x = [word_index[word.lower()] for word in story]
30         xq = [word_index[word.lower()] for word in ques]
31         a = np.zeros(len(word_index) + 1)
32         a[word_index[ans]] = 1
33         X.append(x)
34         Xq.append(xq)
35         A.append(a)
36     return pad_sequences(X, maxlen=max_story_len), pad_sequences(Xq, maxlen=max_ques_len),
np.array(A)
37 input_train, queries_train, answers_train = vectorize_data(train_data)
38 input_test, queries_test, answers_test = vectorize_data(test_data)
39 input_sequence = Input((max_story_len,))
40 ques_sequence = Input((max_ques_len,))
41 input_encoder_m = Sequential()
42 input_encoder_m.add(Embedding(input_dim=vocab_len, output_dim=64))
43 input_encoder_m.add(Dropout(0.2))
44 input_encoder_c = Sequential()
45 input_encoder_c.add(Embedding(input_dim=vocab_len, output_dim=max_ques_len))
46 input_encoder_c.add(Dropout(0.2))
47 ques_encoder = Sequential()
48 ques_encoder.add(Embedding(input_dim=vocab_len, input_length=max_ques_len, output_dim=64))
49 ques_encoder.add(Dropout(0.2))
50 input_encoded_m = input_encoder_m(input_sequence)
```

```
51 input_encoded_c = input_encoder_c(input_sequence)
52 question_encoded = ques_encoder(ques_sequence)
53 match = dot([input_encoded_m, question_encoded], axes=(2, 2))
54 match = Activation('softmax')(match)
55 response = add([match, input_encoded_c])
56 response = Permute((2, 1))(response)
57 answer = concatenate([response, question_encoded])
58 answer = LSTM(32)(answer)
59 answer = Dropout(0.5)(answer)
60 answer = Dense(vocab_len)(answer)
61 answer = Activation('softmax')(answer)
62 model = Model([input_sequence, ques_sequence], answer)
63 model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
64 model.summary()
65 history = model.fit([input_train, queries_train], answers_train,
66                     validation_data=([input_test, queries_test], answers_test),
67                     batch_size=30,
68                     epochs=15)
69 loss, accuracy = model.evaluate([input_test, queries_test], answers_test)
70 print("Test Loss:", loss)
71 print("Test Accuracy:", accuracy)
72 model.save("chatbot_model.h5")
73 def predict_answer(input_text, question_text):
74     input_seq = tokenizer.texts_to_sequences([input_text])
75     input_seq = pad_sequences(input_seq, maxlen=max_story_len)
76     ques_seq = tokenizer.texts_to_sequences([question_text])
77     ques_seq = pad_sequences(ques_seq, maxlen=max_ques_len)
78     prediction = model.predict([input_seq, ques_seq])
79     predicted_word_index = np.argmax(prediction)
80     for word, index in tokenizer.word_index.items():
81         if index == predicted_word_index:
82             return word
83 input_text = input()
84 question_text = input()
85 print("Predicted answer:", predict_answer(input_text, question_text))
```